EEP 596 Computer Vision

HW #2

The purpose of this homework is to continue to explore basic image processing and computer vision concepts using Python and associated libraries. You are free to work with fellow students but should turn in your own work. Specifically, you should be able to explain anything in your assignment if asked.

You should turn in your assignment as “Assignment2.py” and “Report.pdf” in Gradescope. You should develop code with the skeleton provided. You can use print() and cv2.imwrite() to help you to generate outputs for the report, but these should be commented while turning in.

1. **Floodfill.** Implement the stack-based floodfill algorithm as a Python function named floodfill. The function should take as input a seed and return an image after performing the stack-based floodfill algorithm. Perform the algorithm on the 'ant_outline.png' image and fill the ant’s face with red color. (Why would you not want to apply floodfill to a JPEG image?)
   Details: 1) Load the image as color (BGR). 2) Fill with BGR (0, 0, 255). 3) Use 8-connected neighbors (including diagonals). 4) The seed is specified as (x, y), corresponding to image[y, x].

2. **Convolution for Gaussian smoothing.** Write a function to convolve a grayscale image with the 2D separable Gaussian kernel defined by the 1D kernel, 0.25 * [1 2 1]. Implement this as two 1D convolutions with a vertical 3×1 kernel and a horizontal 1×3 kernel. Apply the function to the image `cat_eye.jpg`. Repeat the convolution **5** times in succession to generate increasingly blurrier outputs. Store the blurred images sequentially in a list and return. You should write the convolve operation with only add and multiply operations and a double for-loop over the pixels, without calling existing functions (e.g., np.convolve).
   Details: 1) Load the original image as single-channel uint8. 2) Returned images should have the same shape as the original image. 3) Use floating-point arithmetic for intermediate computations to avoid rounding errors during repeated smoothing. The output images may retain floating-point precision. 4) Use zero padding.

3. **Convolution for differentiation along the vertical direction.** Repeat the previous problem with a 2D Gaussian vertical derivative, defined along the horizontal direction by the 1D smoothing kernel, 0.25 * [1 2 1], and along the vertical direction by the 1D differentiating kernel, 0.5 * [1 0 -1]. (This is the Sobel kernel.) In this case, do **not** repeat the convolution 5 times in succession. Rather, apply the convolution to each of the 5 smoothed images from the previous problem. Store the images in a list and return. You should write the convolve operation with only add and multiply operations and a double for-loop over the pixels, without calling existing functions (e.g., np.convolve).
   Details: 1) Use each of the five smoothed images returned by Task 2 as input. Intermediate calculations may use floating-point arithmetic. 2) Returned images should have the same shape as the original image. 3) Returned images should also be single-

*channel uint8. 4) Because computation involves negative values, you will need to use signed ints (or floats) for computation. 5) To convert back to uint8, use the following formula: p_{out} = clamp(2 * p_{in} + 127) (the scale factor of 2 increases visibility, adding an offset of 127 preserves negative values after conversion, and clamping avoids wraparound). 6) Use zero padding. 7)Do not flip the kernel. Apply it in the order written above.*

4. **Differentiation along another direction along the horizontal direction.** Repeat the previous problem with the transpose of the Gaussian derivative, i.e., defined along the vertical direction by the 1D smoothing kernel, 0.25 * [1 2 1], and along the horizontal direction by the 1D differentiating kernel, 0.5 * [1 0 -1]. Store the images in a list and return. As before, you should write the convolve operation with only add and multiply operations and a double for-loop over the pixels, without calling existing functions (e.g., np.convolve).
   Details: Same as previous.

5. **Gradient magnitude.** Write a function to compute the gradient magnitude of an image. Instead of Euclidean norm (which involves square root), use the Manhattan formula, abs(g_x) + abs(g_y), where g_x and g_y are the horizontal and vertical derivatives, respectively. Compute the gradient magnitude using the horizontal and vertical derivative images generated in Tasks 3 and 4. Since these images have been scaled and offset to fit within uint8, convert them back to approximate signed gradient values using (pixel_value - 127) / 2 before calculating the Manhattan magnitude, abs(gx) + abs(gy).
   Repeat this for each of the five smoothed images. Store the gradient magnitude images in a list and return.
   *Details: 1) Returned images should have the same shape as the original image. 2) Returned images should be single-channel uint8. 3) Use floating-point arithmetic for the calculation. 4) Convert the result using pout = clamp(4 * (abs(gx) + abs(gy))).*

6. **Built-in convolution.** Repeat Task 3 (vertical Gaussian derivative) using `scipy.signal.convolve2d`. Apply the operation to each of the five smoothed images from Task 2 and return the results as a list.
   Details: 1) Use zero padding. 2) Maintain the original image dimensions. 3) Use the same scaling, offset, and clamping as Task 3. 4) Note that scipy.signal.convolve2d flips the kernel, so the derivative direction may be reversed compared with Task 3.

7. **Repeated box filtering.** Write a function to repeatedly convolve a 1D box filter [1,1,1] with itself. (Ignore the scaling factor.) The function should take the number of repetitions (i.e., number of convolutions) as input and return the convolved filter. (For example, zero returns the original filter, one performs one convolution, etc.) For each convolution step, the function should return a shape of (N+M-1), where M and N are the lengths of two arrays you compute convolution with. You may implement the full convolution manually or use `np.convolve` with `mode='full'`. The function should return the original filter when num_repetitions = 0 and perform the specified number of convolutions otherwise.
   Details: 1) Be sure to use ints (or floats), in case values go beyond 255.